

GeoRescue: 4-Day Hackathon Development Plan

This document outlines a structured, four-day sprint to build the **GeoRescue / OmniGIS Agent** for the AMD Developer Hackathon. The workload is distributed across four team members to ensure parallel development of the AI agents, spatial logic, GPU infrastructure, and user interface.

1. Team Roles & Responsibilities

Role	Focus Area	Key Technologies
Member 1: AI Orchestrator (Team Lead)	Agent framework setup, prompt engineering, and pipeline flow.	CrewAI / LangGraph, Python
Member 2: GIS & Data Engineer	Data sourcing, API integration, and spatial operations (routing).	GeoPandas, OSMnx, REST APIs
Member 3: ML / Hardware Engineer	Model hosting on AMD MI300X, ROCm optimization, Vision AI.	AMD Developer Cloud, vLLM, Qwen-VL
Member 4: UI & Deployment Engineer	Frontend dashboard, interactive maps, and Hugging Face deployment.	Streamlit, Folium, Hugging Face Spaces

2. Day-by-Day Execution Plan

Day 1: Infrastructure Setup & Scaffolding

- Member 1 (Orchestrator): Initialize the Git repository. Scaffold the basic

CrewAI/LangChain agent structure using mock data to establish the handoff between the Data, Vision, and Spatial agents.

- **Member 2 (GIS):** Write Python scripts to fetch sample pre- and post-disaster satellite imagery (e.g., from Sentinel Hub) and download local road network vector data via OSMnx.
- **Member 3 (ML):** Provision the AMD Instinct MI300X instance on the AMD Developer Cloud. Install ROCm and set up a vLLM server to host a base Llama-3 model.
- **Member 4 (UI):** Create a basic Streamlit app skeleton. Implement a mock Folium map to ensure rendering works correctly inside the Streamlit environment.

Day 2: AI Integration & Spatial Logic

- **Member 1 (Orchestrator):** Connect the agent framework to the live Llama-3 API hosted by Member 3. Refine system prompts for the Spatial and Reporting agents.
- **Member 2 (GIS):** Develop the core spatial overlay function: taking a polygon (simulated flood zone) and intersecting it with the road network to return impassable roads.
- **Member 3 (ML):** Deploy the multimodal Qwen-VL model on the AMD MI300X. Create an API endpoint that accepts a satellite image and returns a GeoJSON polygon of the detected damage area.
- **Member 4 (UI):** Build the user input interface (chat/text box for natural language commands). Connect the UI to the agent orchestrator script.

Day 3: Pipeline Integration & End-to-End Testing

- **Member 1 (Orchestrator):** Finalize the agent handoffs. Ensure the Vision Agent's output (damage polygons) correctly feeds into the Spatial Agent's logic.
- **Member 2 (GIS):** Work with Member 4 to ensure the generated safe routes and damaged zones are correctly formatted as GeoJSON for web map rendering.
- **Member 3 (ML):** Optimize inference speeds using ROCm tools. Stress-test the models to ensure they don't timeout during the agentic loop. Record performance metrics for the final pitch.
- **Member 4 (UI):** Integrate the dynamic Folium map into Streamlit. Ensure the UI displays the agent's "thought process" (e.g., "Agent 1 is fetching data... Agent 2 is analyzing images...").

Day 4: Polish, Deployment, and Pitch Prep

- **Member 1 (Orchestrator):** Write the project's Long Description for the submission page, detailing the agent architecture.
- **Member 2 (GIS):** Clean up code, finalize requirements.txt, and write the public GitHub

README detailing how to run the spatial tools.

- **Member 3 (ML):** Draft the "Build in Public" social media posts showcasing the AMD MI300X performance vs. standard hardware.
- **Member 4 (UI):** Deploy the final Streamlit application to Hugging Face Spaces. Verify that API calls to the AMD Cloud models work from the HF Space.
- **All Members:** Record the 3-minute video presentation, take cover images, and submit the project on Lablab.ai before the deadline.

3. Recommended Project Structure

```
georescue-amd-hackathon/
├── agents/           # CrewAI configurations and prompt files
├── data/             # Sample GeoJSON and raster images
├── gis_tools/        # GeoPandas and OSMnx Python scripts
├── ml_serving/       # vLLM and Qwen-VL deployment scripts for AMD
├── app.py            # Main Streamlit UI application
├── requirements.txt  # Project dependencies
└── README.md        # Submission documentation
```